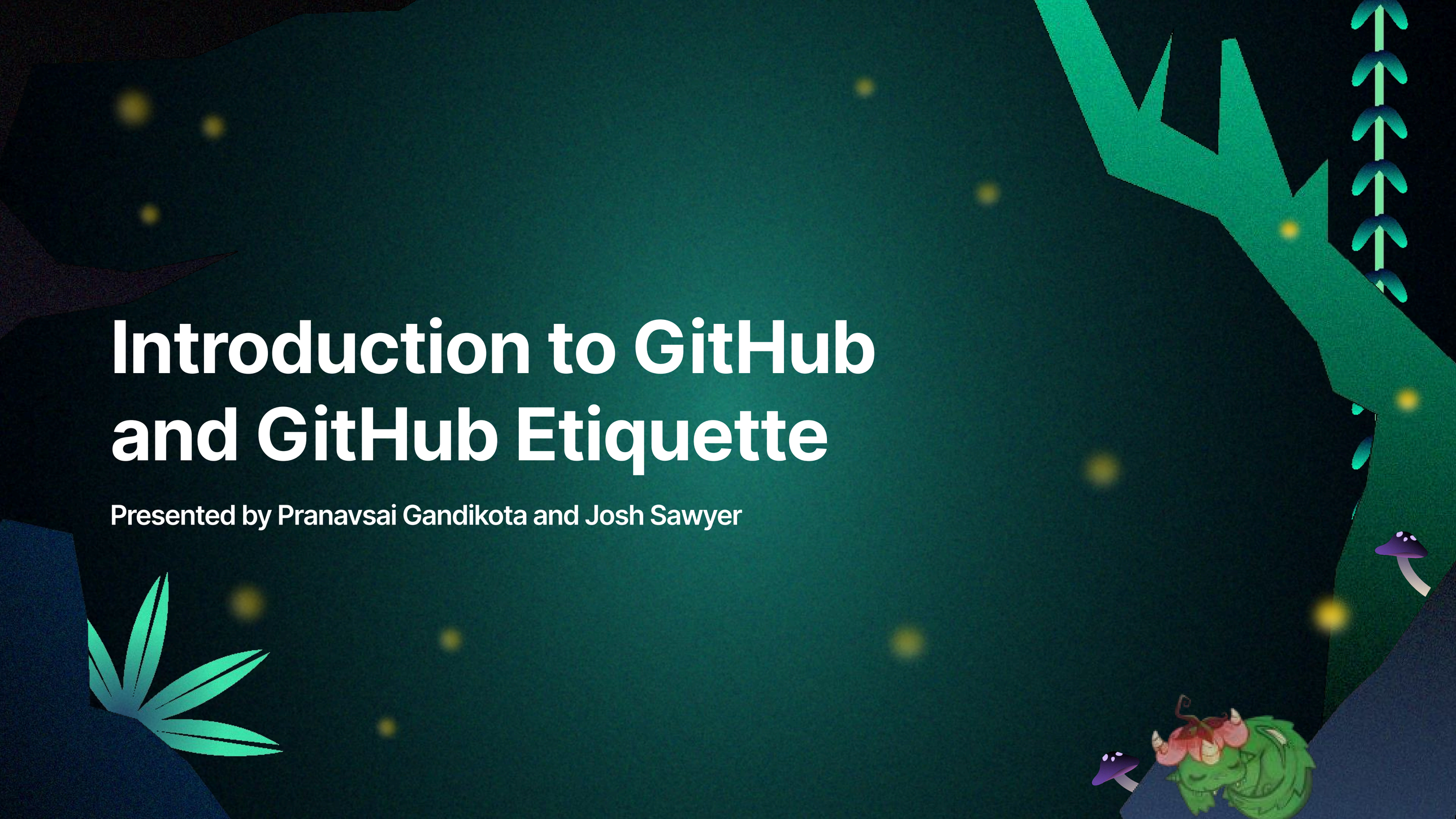
Introduction to GitHub and GitHub Etiquette

Presented by Pranavsai Gandikota and Josh Sawyer



sase Checkin



What You Will Learn

By the end of this workshop, attendees will have a good grasp of:

GitHub

Version control, remotely hosting repositories, and other useful tools for development

Git Basics

How to stage, commit, and upload files to local and remote repositories

The Etiquette

Best practices for Commit messages, Pull Request descriptions, etc.

Working with Others

Apply what you've learned by contributing to a public repository!

What You Will Do

During the activity, attendees will get familiar with:

GitHub Basics

navigating github,
creating repos,
rebasing/pulling,
pushing, cloning, adding
collaborators

GitHub Etiquette

feature branching,
approvals setting github
rules to a repo, creating
issues, Prs

GitHub

Hosting platform for millions of repositories



GitHub

Cloud Platform

Repositories hosted on the cloud
Synchronizes repository for all contributors

Extensive Tooling and Features

Various tools/features offered to improve project development
e.g., GitHub Issues, Pull Requests, GitHub Actions, etc.

Version Control

Tracks changes made to projects over time
Built off of Git

Git != GitHub

Git

Command-line tool that tracks changes made within a local repository



GitHub

Houses Git repositories on the cloud and provides a variety of extra tools to make development easier



Accessing an Existing Repository on GitHub

Cloning

Copying a remote repository to your local device



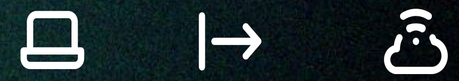
Forking

Creating a remote copy of a remote repository; your own version of an existing project



Pull Requests (PRs)

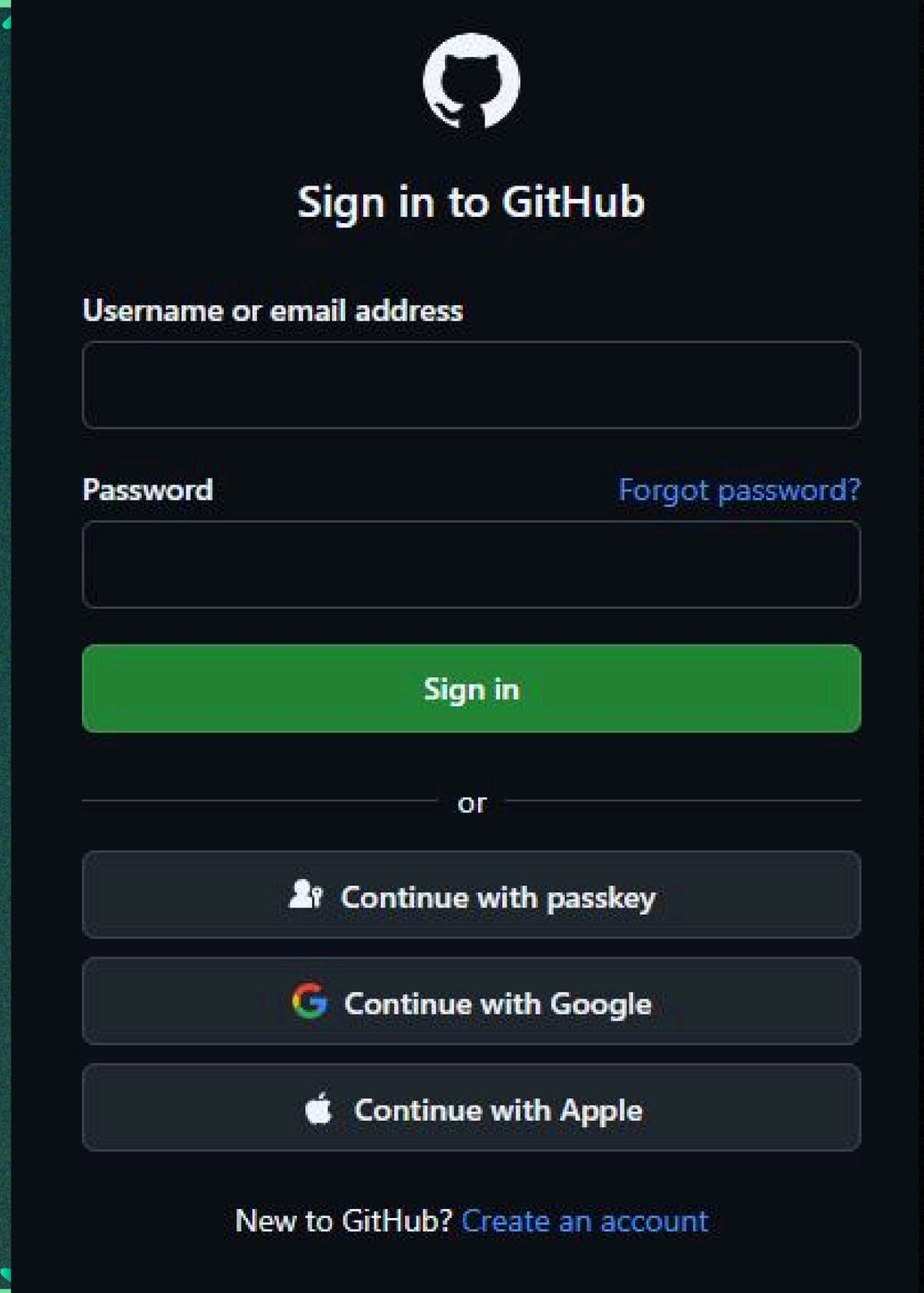
Merging different branches together, allows others to review code before merging

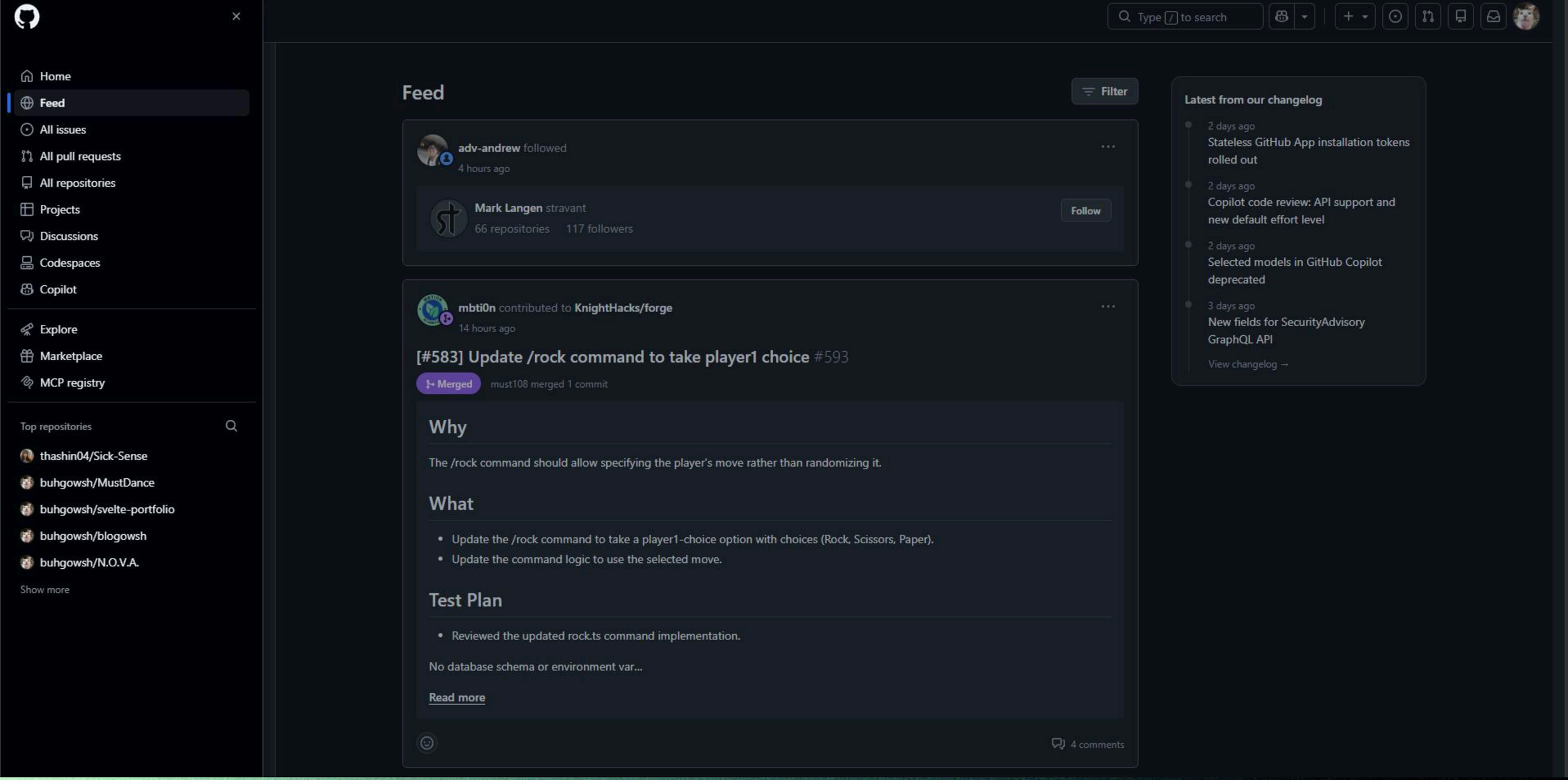


Creating an Account

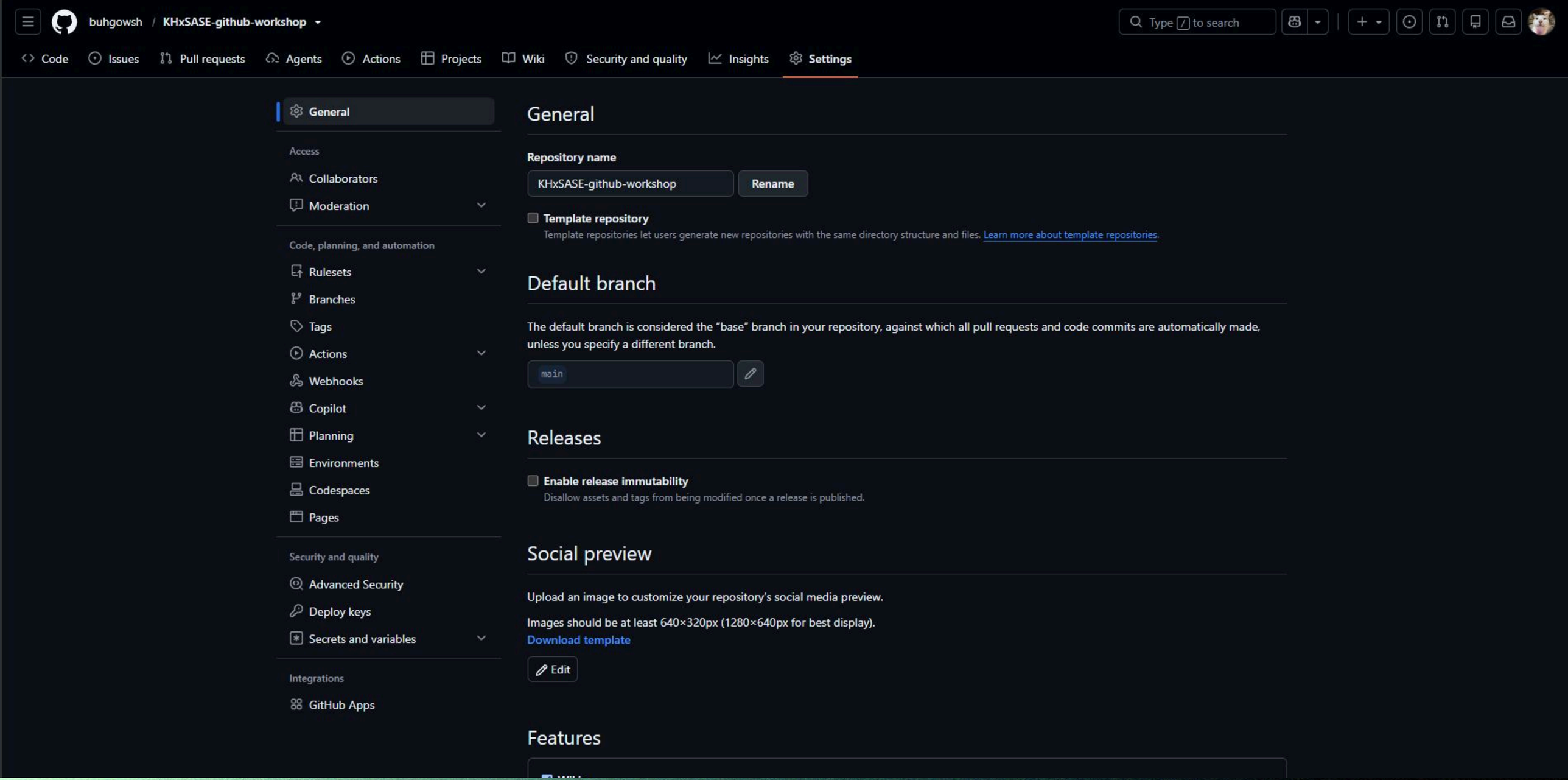
Navigating GitHub

Add your student email to qualify for the GitHub Student Developer Pack!!

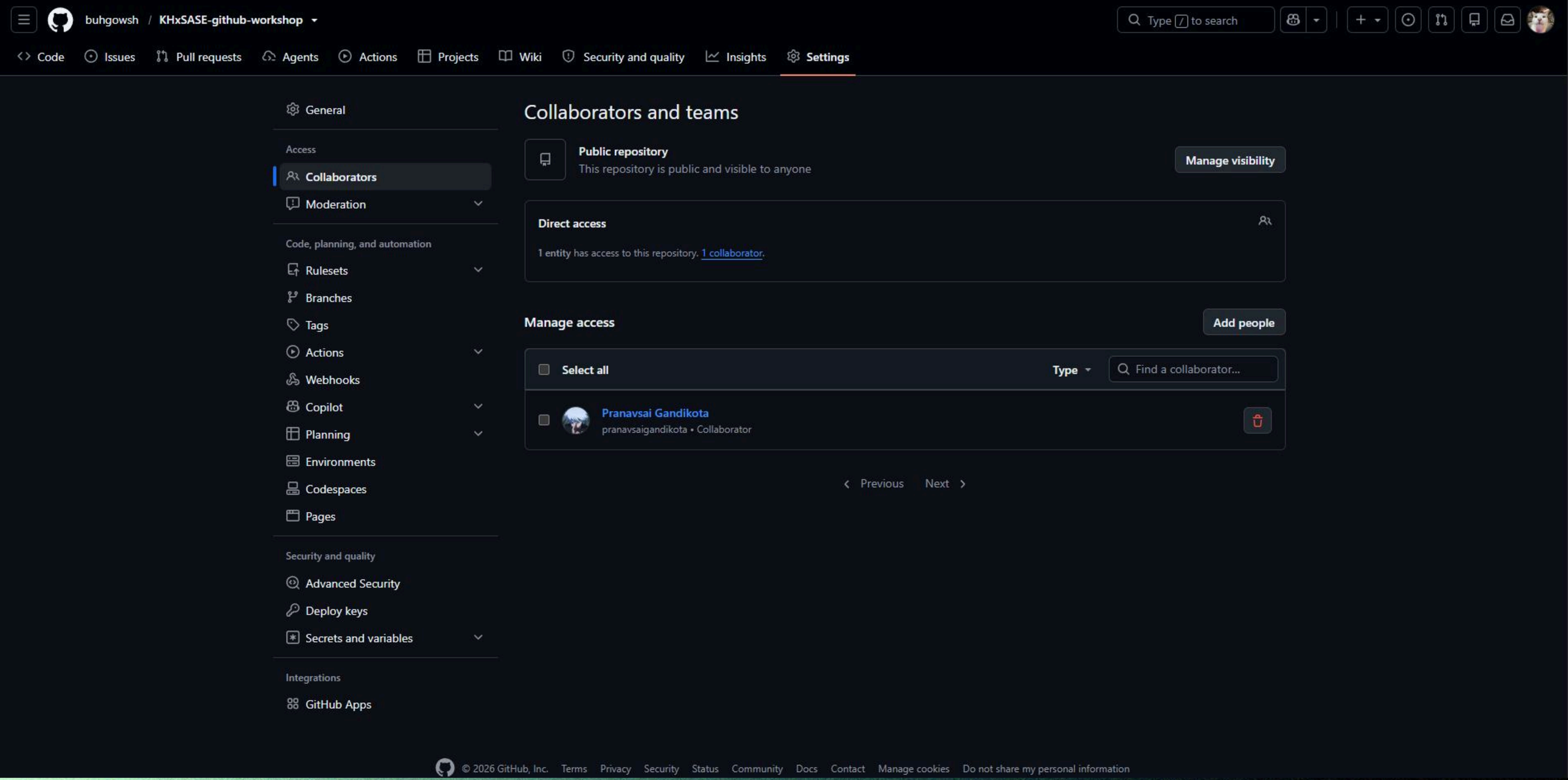




GitHub Feed Page



Repository Settings Page



Collaborators Page

Git Basics

Working with repositories on the command-line





Git Reference Sheet

Every command you'll need with a description of what it does and how to use it

https://shorturl.at/PcUHI



Core Components of Working with Git for GitHub

Stage Changes

Add contents of new/ changed files to the staging area; git add



Commit to Local Repo

Record changes to the repository; git commit



Push to Remote Repo

Update remote repository from local repository; git push



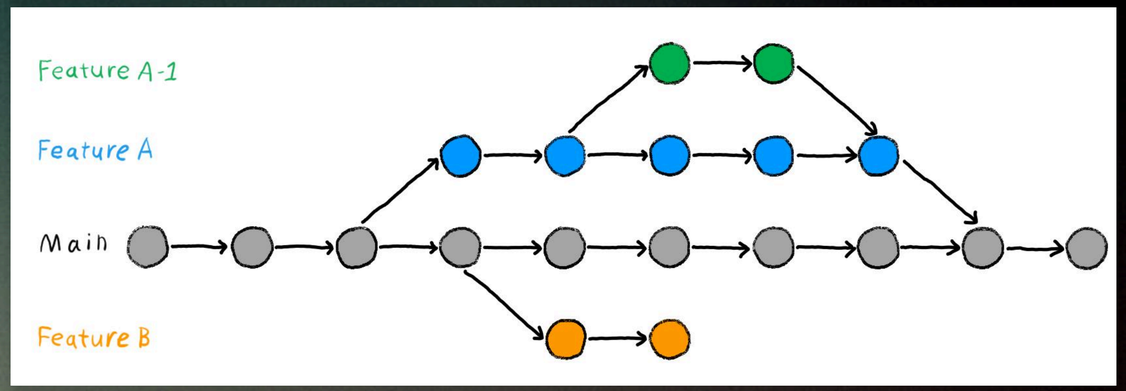
Pull from Remote Repo

Update local repository from remote repository; git pull



Branches

Allows a repository to have multiple versions in parallel; can also merge branches together



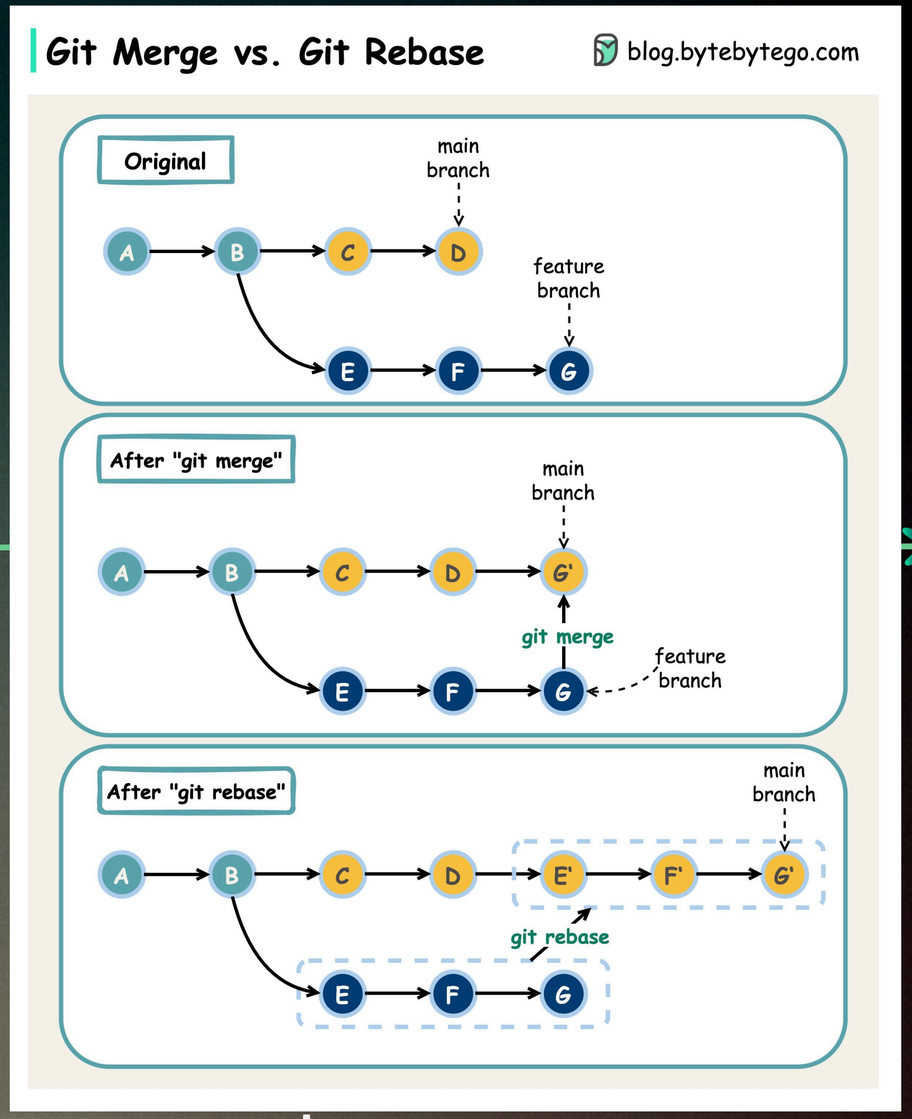
The Version Control Book
lennartwittkuhn.com/version-control-book/

Merging

Combines a series of commits from one branch to the other and replays them all at once as a merge commit; preserves the original commit history and safest

Rebasing

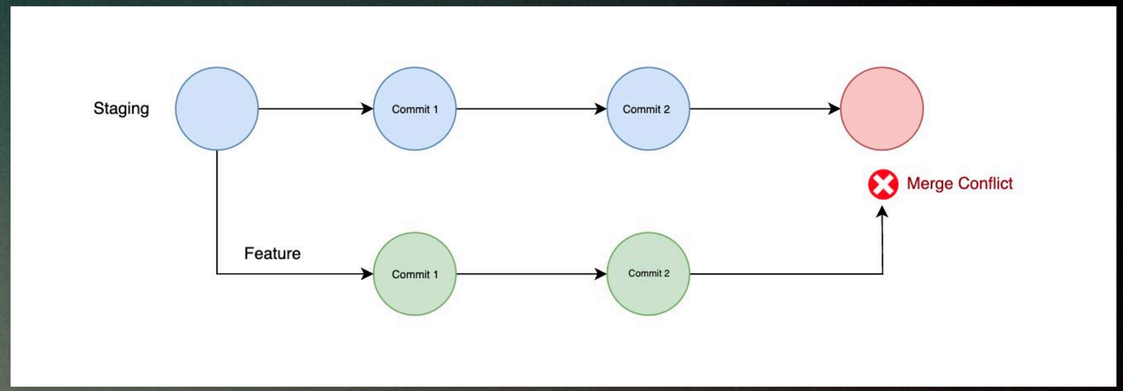
Repeats a series of commits onto a different starting point; creates a clean commit history but does so by rewriting it



ByteByteGo | Git Merge vs. Git Rebase

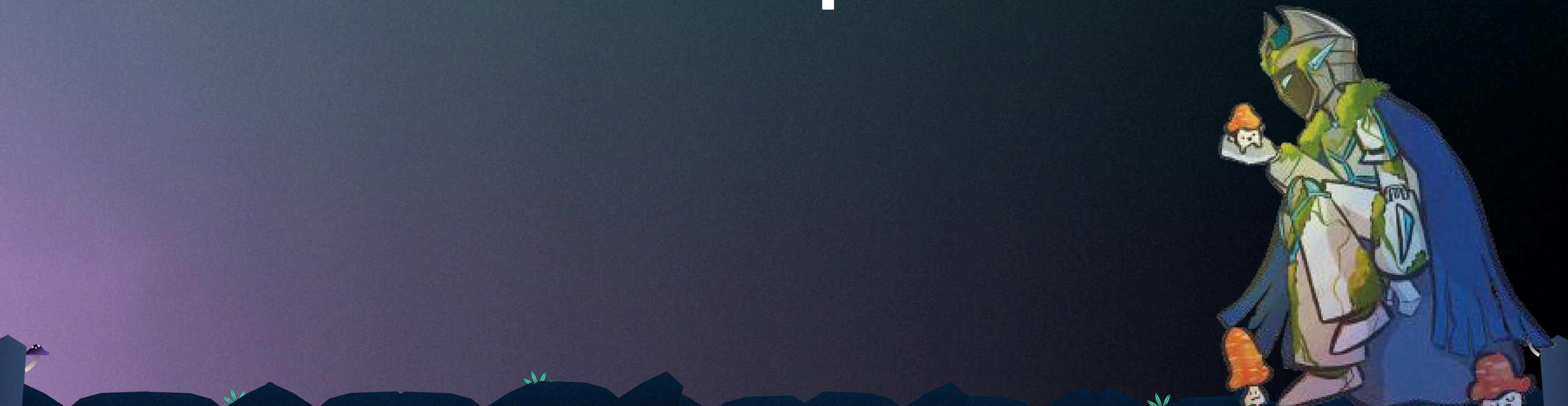
Merge Conflicts

When the local and remote repos have differences that cannot be easily fixed; edit the files causing issue then reattempt the change



Resolve Merge Conflicts in Git | Appsmith
docs.appsmith.com/advanced-concepts/version-control-with-git/commit-and-push

The Etiquette



Commits

```
git add .
git commit -m "Your descriptive commit message"
git push
```

Writing a Good Commit Message

Subject-Body Format

Keep subject under 50 characters, add body after a blank line if needed

Stay Consistent

If your team agrees on a convention, stay within it

Follow Conventional Commits

Start subjects with “fix:” “feat:” “docs:” etc.

Focus on Why

Explain why the changes were made, instead of what was changed

Commit Cheat Sheet



Dont Do

```
* 7d0fc3e typo
* 8fc509a more changes
* efe5fc5 add test
* 447c5a0 updates
* 1189cf0 update condition
* 68abca0 updates
* 29f73ed more changes
* cefaa18 add file
```



Do

```
➜  ng-poopy  master ✗  git add .
➜  ng-poopy  master ✗  git cz

All commit message lines will be cropped at 100 characters.

? Select the type of change that you're committing: (Use arrow keys)
❯ feat:     A new feature
  fix:      A bug fix
  docs:     Documentation only changes
  style:    Changes that do not affect the meaning of the code
            (white-space, formatting, missing semi-colons, etc)
  refactor: A code change that neither fixes a bug or adds a feature
  perf:     A code change that improves performance
  test:     Adding missing tests
  chore:    Changes to the build process or auxiliary tools
            and libraries such as documentation generation
```

```
fix: prevent racing of requests

Introduce a request id and a reference to latest request. Dismiss
incoming responses other than from latest request.

Remove timeouts which were used to mitigate the racing issue but are
obsolete now.

Reviewed-by: Z
Refs: #123
```

```
feat!: drop support for Node 6

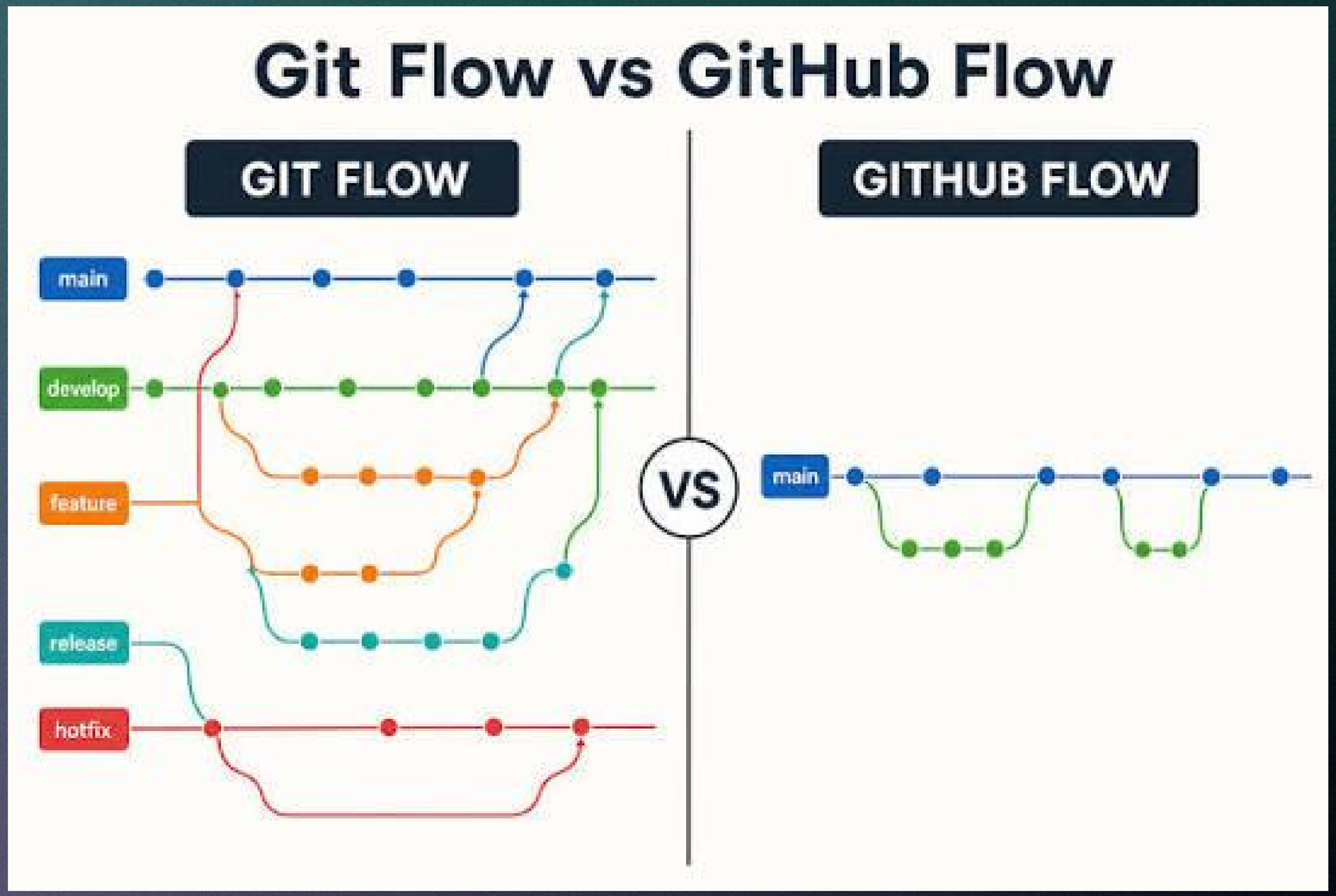
BREAKING CHANGE: use JavaScript features not available in Node 6.
```

```
docs: correct spelling of CHANGELOG
```

Examples of good commits

Branching

Different Flows (that you might use)



https://github.com/bharatdwarkani/gitflow-example

Different Flows (other ones)

Industry / Company Type	Typical Branching Flow	Real-World Examples	Why They Use It
Big Tech & Scale-ups	Trunk-Based Development (TBD)	Google, Meta, Netflix, Amazon	Extreme velocity and continuous delivery. Short-lived branches and feature flags eliminate merge conflicts.
SaaS & Tech Startups	GitHub Flow	Shopify, GitHub	Lightweight. Uses short-lived feature branches with immediate deployment after Pull Request approval.
Enterprise & Regulated	GitFlow / Release Branching	Finance, Healthcare, Aerospace	Requires strict compliance, audit trails, and manual QA for rigid, scheduled release windows.
Hybrid / Staged Systems	GitLab Flow	Cloud Apps, Mobile Apps	Changes flow through sequential environments (dev → staging → prod) to guarantee stability.
Open Source Projects	Forking Workflow	Linux Kernel, Apache Foundation	Allows external contributors to propose changes via Pull Requests without having direct write access to the main repository.

Pull Requests

gh pr create --base main --head feature-branch-name --title "Your PR Title" --body "Your PR description"

Pull Requests (PRs)

Follow Conventional Commits

PR titles only have a single line, use “!” for breaking changes. ex:

feat(checkout): add Apple Pay button
fix(api): resolve timeout on login
feat(api)!: remove legacy v1 endpoints

Give a Detailed Description

Summary of changes, background info and pictures(ss) for visual changes

Review Your Code Before Your Peers

Leave comments in your code asking questions and noting interesting things to the reviewer

Pull Requests

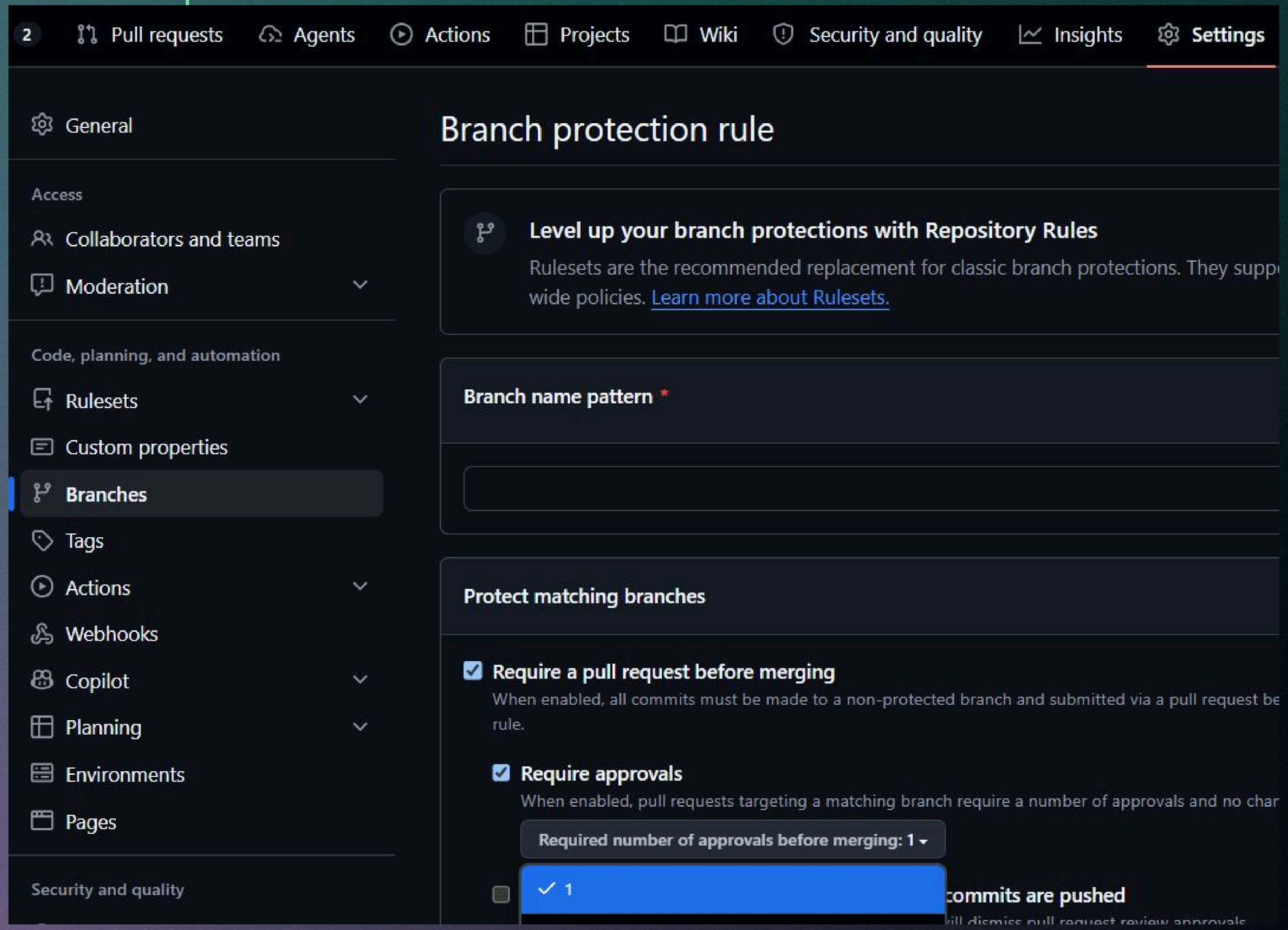
Comments.



Pull Requests

Feature	Commit Message	Pull Request Title
Scope	Captures small change (e.g., one component, one fixed logic loop).	Captures the entire combined feature or fix containing multiple commits.
Example	refactor(auth): extract token extraction logic to helper utility	feat(auth): add MFA verification step during user onboarding (#402)
Purpose	Code history tracking and easy rollbacks.	Code review tracking, project management, and automated release triggers.

Rules and Approvals


2
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
General
Access
Collaborators and teams
Moderation
Code, planning, and automation
Rulesets
Custom properties
Branches
Tags
Actions
Webhooks
Copilot
Planning
Environments
Pages
Security and quality
Branch protection rule
Level up your branch protections with Repository Rules
Rulesets are the recommended replacement for classic branch protections. They supp
wide policies. Learn more about Rulesets.
Branch name pattern *
Protect matching branches
Require a pull request before merging
When enabled, all commits must be made to a non-protected branch and submitted via a pull request be
rule.
Require approvals
When enabled, pull requests targeting a matching branch require a number of approvals and no char
Required number of approvals before merging: 1
1
commits are pushed

Some more things to keep in mind...

DONT PUSH .env

you can use a .gitignore for this to block unnecessary files

DONT FORCE

avoid force pushing to prevent loss of work.

git pull

before you start working to make sure your always updated

README.md

Explain what the project is, how to install it, and how to run it so others can understand what you did

Working With Others

Time to apply what you've been learning!

1. Fork the repo and add your group as collaborators
2. Each collaborator will create a branch to edit anything
3. merge into forked main with a pull request after pulling from main
4. merge into main

Collaboration Repository



buhgowsh/KHxSASE-github-workshop

Extra Tools/Features

Quick shoutout to other useful things GitHub offers

More Things GitHub Offers

Tools

GitHub Pages

Turns repositories into live websites

GitHub Packages

Platform for hosting packages like containers and other dependencies

GitHub Student Developer Pack

Provides courses to continue learning and additional benefits across a variety of services

Features

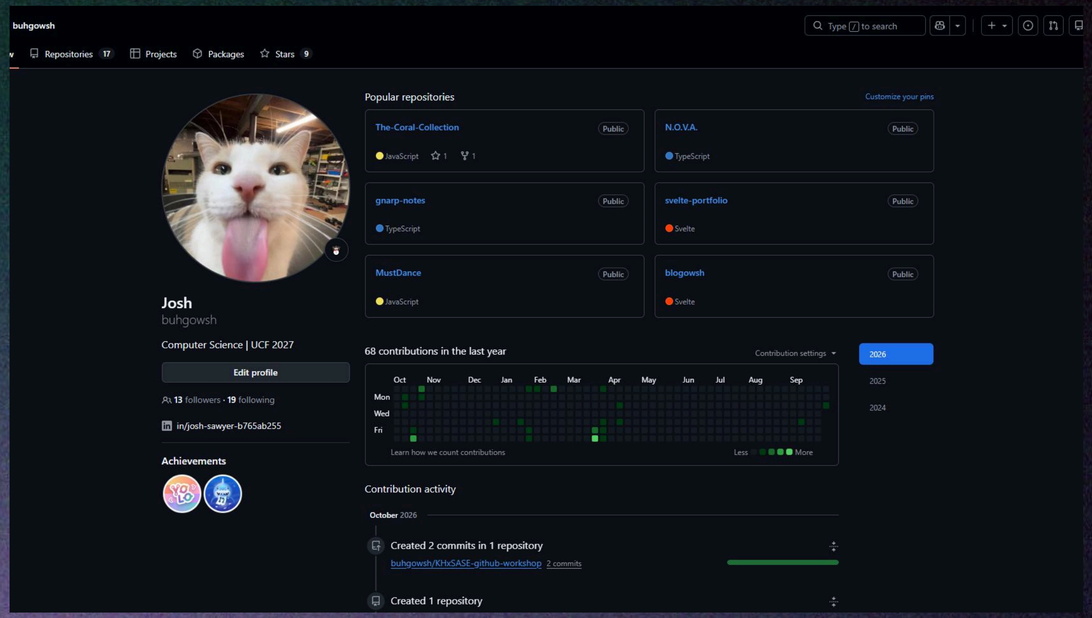
GitHub Actions

Automate workflows with CI/CD

GitHub Issues

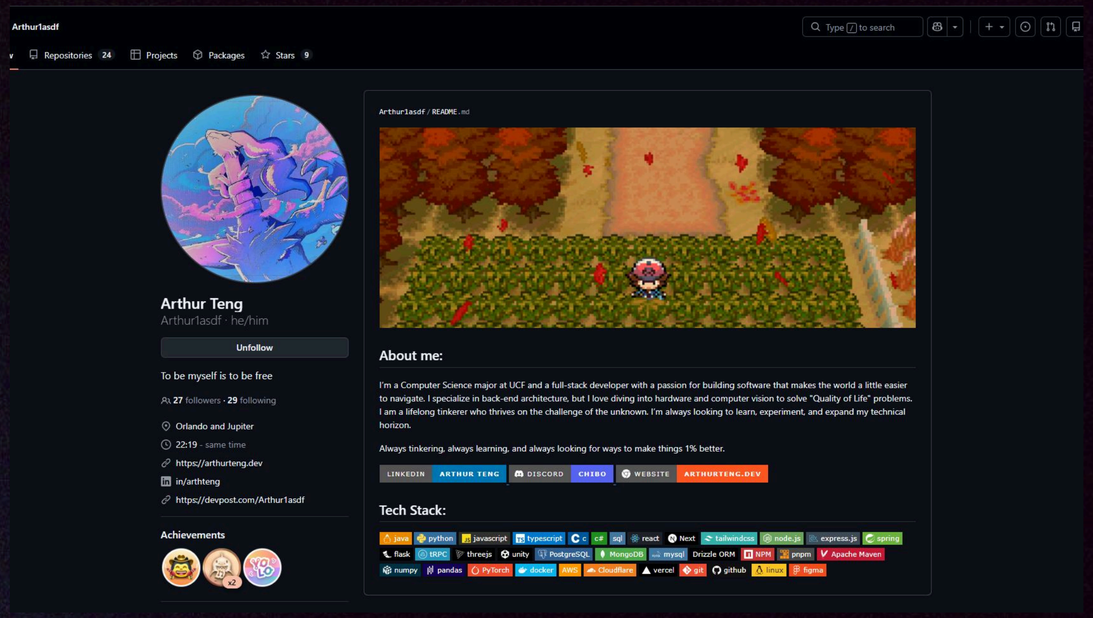
Used to plan, discuss, and track work within a repository

Profile Decoration



Minimal Detail

Achievements, repositories, contributions, and contribution activity



Refined Profile

Stylized profile showcasing interests and skills